Spatial Screening and Reinforcement Learning-Based Particle Swarm Optimization for Real-Time or Resource-Constrained Engineering Optimization

Qing-Wei Chai, Yan Xia, Liang Qi, *Senior Member, IEEE*, and Jeng-Shyang Pan, *Senior Member, IEEE*

***Abstract*—Traditional swarm intelligence algorithms rely on exhaustive full-population evaluation, which expends redundant computational resources on spatially redundant particles and restricts their deployment in real-time or resource-constrained engineering systems. Rather than pursuing the highest attainable accuracy, this paper seeks to preserve acceptable accuracy with fewer fitness evaluations and shorter wall-clock runtime, thereby broadening the application scenarios of swarm intelligence algorithms. To this end, we propose a Spatial Screening and Reinforcement Learning-Based Particle Swarm Optimization (SS-RLPSO). A spatial screening mechanism evaluates a sparse subset of representative particles, and the saved evaluations sustain additional generations, keeping the algorithm competitive while using only 20% of the standard budget. To compensate for the resulting information loss, a reinforcement learning agent schedules four dedicated operators according to the evolutionary phase and stagnation status of the swarm. Experiments on the CEC2017 benchmark suite and the CEC2013 large-scale global optimization (LSGO) suite show that SS-RLPSO attains accuracy comparable to representative PSO variants. With only 20% of the budget, it still matches full-budget peers while consuming one-third to one-half of their runtime; on 1000-dimensional problems, its accuracy trails that of the decomposition- and surrogate-based large-scale optimizers, but its runtime is only one-tenth to one-thousandth of theirs. Ablation studies attribute the main gain to spatial screening, with the RL scheduler mainly helping on complex multimodal functions, and sensitivity analysis confirms that screening 90% of particles is robust. These results indicate a favorable trade-off between accuracy and efficiency for real-time or resource-constrained engineering tasks.**

***Index Terms*—Particle swarm optimization (PSO), sparse evaluation, spatial screening, reinforcement learning, operator scheduling, expensive optimization, large-scale global optimization.**

I. Introduction

OVER the past three decades, metaheuristic algorithms have been established as a cornerstone for solving complex optimization problems across diverse engineering domains [1]. Representative paradigms such as particle swarm optimization (PSO) [2]–[4] and differential evolution (DE) [5]–[7] are widely adopted in practical applications including production job shop scheduling, mobile robot path planning, and large-scale water distribution network design [8]–[10].

However, real-world engineering applications are becoming increasingly sophisticated, and many of them rely on computationally expensive simulations within strictly limited decision time. Under such resource-constrained conditions, a metaheuristic must reconcile three conflicting requirements to perform well on complex optimization problems: preserving population diversity, reducing costly fitness evaluations, and adapting the search behavior to the problem landscape. Early research concentrated on population topology design, which introduces structured learning rules like comprehensive learning and multi-group partitioning to control information exchange and maintain global exploration capacity [11]–[13]. Although these approaches can delay premature convergence, they rely on fixed, manually designed interaction rules throughout the search process, making it difficult to respond to dynamic shifts in the objective space. Competitive frameworks, including the Competitive Swarm Optimizer (CSO) and its variants, were later proposed to add dynamic search incentives [14], [15]. These algorithms create comparative relations between individuals or subpopulations to strengthen local exploitation, yet they may reduce population diversity excessively. Recent adaptive and learning-aided optimization methods offer automated ways to tune parameters and select evolutionary operators [16]–[18]. Nevertheless, such techniques usually require heavy offline training or manually designed reward functions tailored for specific problems. Overall, most existing improvements optimize individual components of the search system in isolation. How to integrate efficient population evolution, dynamic resource allocation, and adaptive algorithmic control within a unified collaborative framework remains an open research gap.

Most algorithms covered above adopt an exhaustive population evaluation rule that imposes full Fitness Evaluation (FE) calculation on every individual per generation, ignoring differences in each particle's search location and evolutionary potential [19]. This fixed evaluation mode wastes part of the FE budget on overlapping search zones and low-value local regions, resulting in poor utilization of limited computing resources [20], [21]. To mitigate this waste, surrogate-assisted evolutionary algorithms (SAEAs) train cheap approximation models, such as Gaussian processes and radial basis functions, from previously evaluated solutions and use model predictions

Q.-W. Chai and L. Qi are with the School of Computer Science and Engineering, Shandong University of Science and Technology, Qingdao 266590, China, and also with the Shandong Key Laboratory of Smart Mine Information Technology, Qingdao 266590, China (e-mail: mimanxiaowei@163.com; qiliang@sdust.edu.cn).

Y. Xia is with the School of Computer Science and Engineering, Shandong University of Science and Technology, Qingdao 266590, China (e-mail: xiayansdust@163.com). (Corresponding author: Yan Xia.)

J.-S. Pan is with the School of Artificial Intelligence and the School of Future Technology, Nanjing University of Information Science and Technology, Nanjing 210044, China (e-mail: 100112@nuist.edu.cn).

This work was supported in part by the Shandong Key Laboratory of Smart Mine Information Technology.

to prescreen candidate individuals, so that only a fraction of the population requires real fitness evaluation [19], [21], [22]. However, this path faces a fundamental trade-off in which building precise approximation models requires abundant sample points that defeat the aim of cutting computational costs, while insufficient sampling produces inaccurate predictions that mislead the entire search process. In addition, model training introduces a different computational bottleneck. Building accurate surrogate models demands matrix inversion operations with $O(N_s^3)$ computational complexity, where N_s is the number of samples used to train the model, and occupies a large amount of memory [19], [23]. This extra computation extends the overall running time and compromises the lightweight and fast-running characteristics of metaheuristics, limiting their deployment on industrial hardware with tight computing constraints. Ultimately, neither the traditional exhaustive evaluation framework nor surrogate-aided designs achieve true computational efficiency. To bypass the dual drawbacks of redundant full evaluations and heavy model training, this work develops a lightweight, model-free evaluation strategy that only explicitly evaluates a small subset of representative individuals in each generation.

Therefore, in this article, we propose a Spatial Screening and Reinforcement Learning-Based Particle Swarm Optimization (SS-RLPSO) for solving expensive optimization problems. It should be emphasized that the primary objective of SS-RLPSO is not to surpass state-of-the-art optimizers in terms of final convergence accuracy. Instead, it aims to preserve acceptable solution accuracy while reducing the number of fitness evaluations and the overall wall-clock runtime, thereby extending the applicability of swarm intelligence algorithms to real-time or resource-constrained scenarios where full-evaluation algorithms are computationally unaffordable. In the sparse evaluation mechanism, only representative particles are selected for real fitness evaluation in each generation to save FEs and prolong the evolutionary generations, so as to refine the solution accuracy. Specifically, a spatial screening mechanism is first designed to select a sparse subset of representative particles for real fitness evaluation. It evaluates the individuals closest to the current global best to refine the promising region, together with those farthest from the population centroid to explore under-sampled areas, while the remaining particles are shielded from evaluation. Then, a reinforcement learning agent is embedded to monitor real-time evolutionary status and adaptively switch among search operators, further ensuring that the limited fitness budget is allocated to the most promising regions.

The main contributions of this article are as follows.

1) A model-free sparse evaluation framework that performs on-demand fitness assessment without building any surrogate model, thereby saving fitness evaluations and avoiding the heavy matrix-inversion overhead inherent in surrogate modeling.

2) A lightweight spatial screening mechanism whose distance-based selection costs only $O(D)$ per particle, which evaluates the particles closest to the current global best to refine the promising region, together with those farthest from the population centroid to explore under-sampled regions, while shielding the spatially redundant ones from real fitness evaluation, converting the saved evaluations into additional evolutionary generations.

3) An RL-based operator scheduler that compensates for the performance loss caused by spatial screening. Since screening evaluates only a small subset of particles, the population receives sparse feedback and loses diversity, which may impair search performance; an RL agent then monitors the evolutionary phase and population stagnation status and selects one of four dedicated operators per generation.

4) Systematic experiments on the CEC2017 and CEC2013 LSGO benchmark suites with Wilcoxon rank-sum tests and component-wise ablation and sensitivity analyses demonstrate a favorable accuracy–efficiency trade-off of SS-RLPSO. With only 20% of the standard evaluation budget, it attains accuracy comparable to that of full-budget peer algorithms such as CLPSO at roughly one-third to one-half of their wall-clock time. On 1000-dimensional problems, it preserves acceptable accuracy while being more than two orders of magnitude faster than the surrogate-assisted optimizer LSEO [24], confirming its practicality for real-time and resource-constrained optimization.

II. Related Work

A. Advanced Particle Swarm Optimizers

As a classic swarm intelligence algorithm, PSO updates particles via personal and global best positions, but often suffers from premature convergence in high-dimensional multimodal spaces. To alleviate this issue, researchers have developed various improved PSO variants, which can be roughly divided into two categories based on their core driving mechanisms.

The first category enhances population diversity through elaborate learning topologies. Liang et al. [11] proposed comprehensive learning PSO (CLPSO), in which each dimension of a particle independently selects a learning exemplar from the historical best positions of other particles via a tournament selection strategy, rather than consistently following the single global best. This comprehensive learning strategy improves the exploration capability on multimodal functions, but it requires evaluating a large population to maintain sufficient exemplar diversity. Similarly, Lim and Isa [13] proposed a teaching and peer-learning PSO (TPLPSO), in which the population is partitioned into multiple hierarchical layers according to fitness. In the teaching phase, lower-level particles learn from higher-level ones to accelerate convergence; in the peer-learning phase, middle-level particles learn from better particles within the same layer to maintain diversity. While these topology-based methods enrich information exchange channels, their performance heavily depends on population size, meaning that shrinking the population for expensive optimization will directly degrade their learning effectiveness.

The other category improves search efficiency through adaptive parameter control. Ratnaweera et al. [25] proposed a self-organizing hierarchical PSO with time-varying acceleration coefficients (HPSOTVAC), in which the cognitive coefficient decreases and the social coefficient increases over the search, and particles stagnating in the search space are reinitialized

to escape local optima. Zhan et al. [4] proposed adaptive PSO (APSO), which tracks population distribution entropy and evolutionary factors to distinguish four evolutionary states: exploration, exploitation, convergence, and jumping out. Based on these states, APSO dynamically adjusts inertia weights and acceleration coefficients to adapt the search behavior. Following this idea, Liu et al. [26] proposed a sigmoid-function-based adaptive weighted PSO (AWPSO), which employs a nonlinear sigmoid function to adaptively balance global and local search according to the current evolutionary state. Although these adaptive rules reduce manual parameter tuning, their decision criteria are typically predefined heuristic rules. Moreover, both APSO and AWPSO still evaluate the entire population in each generation, which means their fitness evaluation cost remains unchanged and thus limits their applicability on expensive optimization tasks.

Recent advances in PSO design remain active. Tian et al. [27] proposed a diversity-guided PSO with a multi-level learning strategy (DPSO-MLS), which combines opposition-based initialization, attract-and-repulse search, and several exemplar-based update rules. Yang and Li [28] developed an evolutionary-state-driven multi-swarm PSO (ESD-PSO), which estimates the evolutionary state of each sub-swarm, adaptively regroups sub-swarms, and injects extra samples into stagnant sub-swarms. Moazen et al. [29] presented PSO-ELPM, which adopts elite particles as learning exemplars and couples enhanced parameter updating with an exponential mutation operator. Tang et al. [30] proposed a multi-subswarm cooperative PSO (MSC-PSO), which combines level-based inertia weight adjustment with an adaptive learning strategy. Although these recent variants improve search accuracy on standard benchmark suites, none of them explicitly addresses the expensive-evaluation scenario in which the total evaluation budget is restricted.

Despite their different design principles, the PSO variants above all follow an exhaustive evaluation rule, where each particle is evaluated once per generation. This makes the computational cost scale linearly with population size and dimension, limiting their use on large-scale or simulation-based expensive problems.

B. Large-Scale and Surrogate-Assisted Optimization

Research on reducing the evaluation burden of PSO has followed two lines, large-scale optimization and surrogate-assisted optimization. Both lower the number of real fitness evaluations, as does our method, but the saved evaluations are used differently. Surrogate-assisted methods spend them on approximation models so that expensive optimization problems become solvable within a limited evaluation budget, whereas SS-RLPSO builds no model and converts them into additional evolutionary generations and a shorter wall-clock time.

As problem dimensionality increases, conventional PSO variants encounter the curse of dimensionality [31]. To address this challenge, cooperative coevolutionary frameworks have been introduced into PSO. Li and Yao [12] proposed CCPSO2, which integrates a cooperative coevolutionary framework into PSO. The algorithm decomposes the original high-dimensional problem into multiple low-dimensional subproblems via a random grouping strategy, and dynamically adjusts the subproblem size during evolution. Each subproblem is optimized independently by PSO, and the solutions of these subproblems are combined to form the complete solution. However, the performance of such decomposition-based methods is sensitive to grouping strategies, and identifying variable correlations often consumes additional fitness evaluations.

To avoid the computational overhead of variable grouping, non-decomposition algorithms have been developed for large-scale optimization. Instead of partitioning the problem, these methods reduce the evaluation burden by limiting the number of updated particles per generation. For instance, Cheng and Jin [14] proposed CSO with a random pairwise competition mechanism. Winners move into the next generation directly without being re-evaluated, whereas losers update their velocities by learning from their matched winners. This pairwise competition removes the need to store personal best (pbest) information but still requires N fitness evaluations per generation, where N is the population size, as every particle must be evaluated once to determine the pairwise comparison results. Inspired by this lightweight design, social learning PSO (SL-PSO) [32] permits each particle to learn from one or multiple superior particles randomly sampled from the current population. Without guidance from the global best position, SL-PSO preserves diversity in high-dimensional spaces. Following this direction of updating only a small subset of promising individuals, Yang et al. [33] proposed a segment-based predominant learning swarm optimizer (SPLS), which partitions the population into several segments and updates only the predominant particle of each segment according to predefined learning rules, thereby further reducing the number of updated particles per generation.

More recently, Liu et al. [34] proposed a small-scale learning PSO (SSLPSO) under the sparse evaluation paradigm, which updates at most two representative individuals per generation according to predefined heuristic rules. Although this scheme saves a large portion of fitness evaluations, two issues remain open. First, updating only one or two particles per generation provides sparse population feedback, so the swarm may lose diversity and converge prematurely on complex multimodal problems. Second, the runtime speedup delivered by SSLPSO is smaller than expected. Since its static selection metrics need to be recomputed across the full population every generation, this additional algorithmic overhead partially offsets the computational savings gained from sparse evaluation, limiting the achievable wall-clock speedup. Across these non-decomposition frameworks, the particles to be evaluated are determined by fixed rules, such as random pairwise competition, fixed learning probabilities, or predefined representative selection, and none of them examines whether an evaluation is still informative before it is spent, so the evaluation budget is repeatedly committed to particles whose information is already represented in the swarm.

Another line of work addresses expensive optimization through surrogate-assisted evolutionary algorithms. Zhao et al. [24] proposed an efficient surrogate-assisted optimizer, which constructs Gaussian process models to approximate the fitness

landscape and uses expected improvement to select candidate solutions for real evaluation. Qin et al. [35] further investigated expensive optimization by integrating local surrogate models into an evolutionary framework, where the approximate models are updated incrementally to guide the search toward promising regions. Surrogate-assisted optimization reduces real fitness evaluations, yet building the surrogate model incurs substantial overhead. For large-scale problems, the $O(N_s^3)$ matrix inversion largely offsets the computational savings, while the approximation accuracy also deteriorates in high-dimensional spaces [36].

Apart from surrogate-assisted optimization, lightweight fitness approximation and fitness inheritance strategies have also been investigated to reduce function evaluation costs [21], [37]. Such approaches infer the fitness of newly generated individuals from their spatial proximity to previously evaluated solutions. Although these schemes bring down computational costs, they tend to yield estimation errors on complex multimodal problem landscapes. Unlike these approximation-based alternatives, our spatial screening mechanism avoids fitness estimation entirely. It identifies and skips redundant evaluations purely via spatial topological information. As a result, it incurs no approximation-induced error and imposes only marginal extra computational overhead while saving computational resources.

C. Reinforcement Learning in Evolutionary Computation

Since the foundational introduction of Q-learning [38], value-based reinforcement learning has proven effective in sequential decision-making under uncertain environments. In recent years, Reinforcement Learning (RL) has been increasingly adopted to replace hand-crafted heuristics in evolutionary algorithms. Some studies focus on parameter control, where the RL agent monitors population status and adjusts mutation rates, crossover probabilities, or inertia weights according to real-time feedback [39]. Tian et al. [16] pioneered this direction with a deep RL framework for adaptive parameter control in multi-objective optimization. Yin et al. [17] designed an RL-based adaptation method for PSO that tunes inertia weight and acceleration coefficients online. Liu et al. [40] further developed an RL-driven adaptive PSO that dynamically balances exploration and exploitation. More recently, Shao et al. [18] extended the role of RL to a higher level, using deep RL to automatically generate complete optimization frameworks with minimal human intervention.

RL-based adaptive strategies have also been extended to large-scale optimization scenarios. Wang et al. [41] developed a level-based PSO framework, where an RL agent assigns suitable learning levels to individual particles according to the evolutionary state. On this basis, Wang et al. [42] further constructed a multi-swarm optimizer, using RL to coordinate information interaction across sub-swarms for improved high-dimensional search efficiency.

Despite the performance gains brought by RL-assisted heuristics, all such methods retain the exhaustive population evaluation mechanism inherited from canonical swarm algorithms. The embedded RL module only regulates particle update rules, without changing the fundamental requirement that each individual must be evaluated once per generation. As a result, evaluation costs still scale linearly with population size and problem dimensionality.

More broadly, none of the methods above removes the evaluation cost completely, and each of them leaves a different problem behind. Topology designs and adaptive rules improve information exchange and parameter tuning, yet they evaluate every particle in each generation. Large-scale optimizers update fewer particles per generation, but decomposition-based variants are sensitive to how variables are grouped and spend extra evaluations on finding variable correlations, while non-decomposition variants use fixed rules to decide which particles to evaluate and never check whether an evaluation is still informative. Surrogate-assisted methods make expensive problems solvable within a tight budget, but building the model takes up part of that budget, since matrix inversion costs as much as $O(N_s^3)$, and the model accuracy drops in high dimensions. RL-assisted methods adjust the update rules automatically, but they leave the number of evaluated particles unchanged, so the evaluation cost stays the same. What is still missing is a lightweight framework that reduces the evaluation cost while maintaining the search performance, and this article fills this gap with SS-RLPSO, a Spatial Screening and Reinforcement Learning-Based Particle Swarm Optimization.

III. Proposed SS-RLPSO Algorithm

This section presents the proposed SS-RLPSO framework. We first introduce the overall architecture and the sparse evaluation framework, followed by the motivation for spatial screening. We then detail the dual-potential spatial screening mechanism and the RL-based operator scheduler, and finally provide a comprehensive complexity analysis.

A. Overview of the Proposed Framework

Fig. 1 illustrates the overall architecture and the algorithmic flowchart of the proposed SS-RLPSO. To reduce the computational burden while maintaining a robust search capability under constrained environments, the framework is organized as a dual-layer structure: an upper-level scheduler and a lower-level evaluation filter.

As depicted on the left side of Fig. 1, the algorithm begins with the initialization of N particles. During the main evolutionary loop, the reinforcement learning-based scheduling module serves as the decision unit of the swarm. It reads the current evolutionary phase and stagnation counter as environmental states to query the Q-table, and then assigns one of four operators to update the particle positions, which are the exploitation-biased update, the peer learning update, the Gaussian perturbation, and the opposition-based global jump.

Subsequently, the particles updated by the selected operator enter the spatial screening mechanism. Instead of exhaustively evaluating the whole population, which would rapidly deplete the computational budget, only a sparse subset of K particles ($K \ll N$), consisting of 5% convergence-driven and 5% diversity-driven individuals, is actually evaluated. The fitness

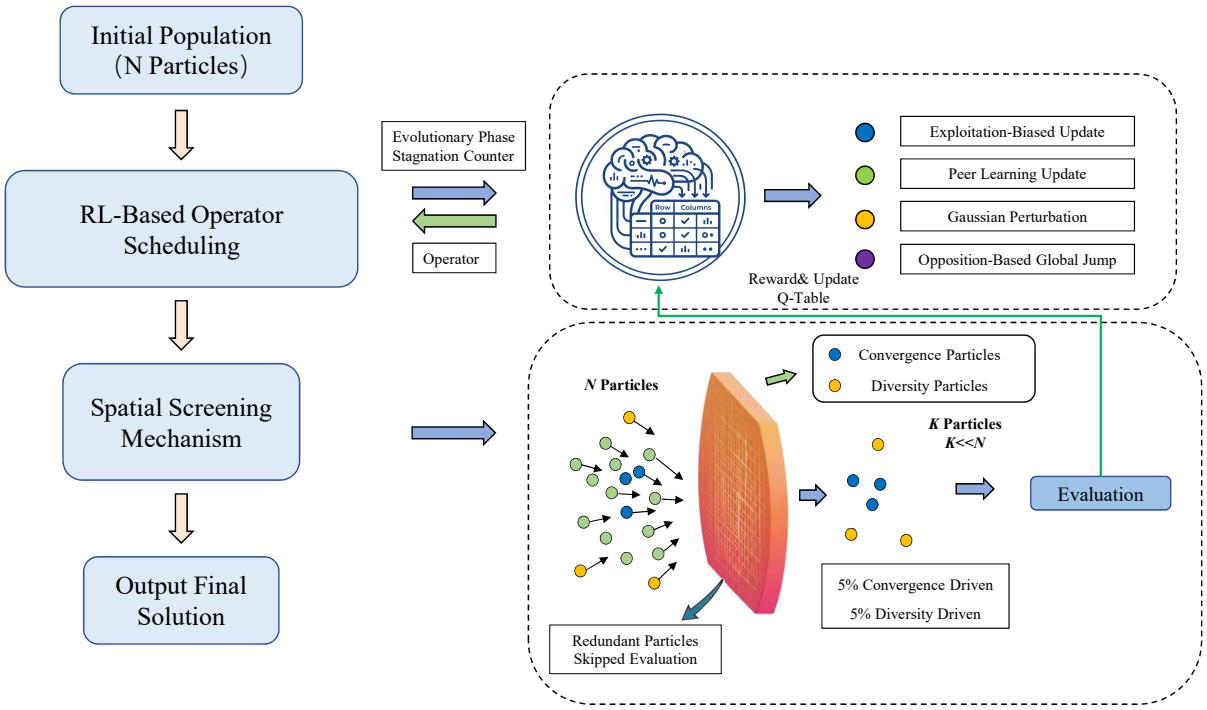


Fig. 1: The overall architecture of the proposed Spatial Screening and Reinforcement Learning-Based Particle Swarm Optimization (SS-RLPSO). The framework mainly consists of an RL-based operator scheduler and a spatial screening mechanism.

improvements obtained from this sparse evaluation serve as the reward signal for updating the Q-table, thereby forming a closed-loop, self-adaptive optimization system.

Algorithm 1 summarizes the complete procedure. It takes the objective function, the problem dimension, the population size, the evaluation budget, and the sparse ratio as inputs, and returns the best fitness found together with the corresponding convergence history. Before the main loop starts, the whole population is initialized and evaluated once, which establishes the initial spatial topology and provides the personal and global bests required by the sparse screening stage. Each subsequent generation then repeats the same sequence of operations until the evaluation budget is exhausted. The agent builds the current state and selects an operator from the Q-table, and all particles are updated with that operator. The screening mechanism then determines the evaluation set from the convergence-driven and diversity-driven distances, and only the particles in this set receive real fitness evaluations that refresh the personal and global bests. Finally, the resulting fitness improvement serves as the reward for the Q-table update on the transition (s, a, s'). As the exploration rate ϵ decays with the consumed budget, operator selection gradually shifts from exploration to exploitation. The spatial screening mechanism and the RL scheduler are detailed in Sections III-C and III-D, respectively.

B. Motivation: A Spatial Redundancy Perspective on Computational Efficiency

As a swarm evolves, its particles gradually gather into a few localized clusters. Particles within the same cluster are close to one another in the search space and, consequently, tend to produce similar fitness values. Evaluating each of them individually thus adds little information, and when a single evaluation is computationally expensive, this spatial redundancy amounts to a waste of the evaluation budget.

Such redundancy follows from the local smoothness of continuous fitness landscapes. If the fitness function f is Lipschitz continuous in the region occupied by the swarm, then there exists a constant $L > 0$ such that for any two particles x_i and x_j:

$$|f(x_i) - f(x_j)| \leq L\|x_i - x_j\| \tag{1}$$

i.e., the fitness difference between two particles is bounded by their spatial distance. As the swarm converges into a promising basin, the distance between neighboring particles becomes small, and so does the expected variation in their fitness. Once a particle has been evaluated, its close neighbors add little new information. This inequality serves as a geometric intuition rather than a rigorous guarantee, because global Lipschitz continuity does not necessarily hold for arbitrary black-box problems. Its practical value lies in motivating the screening strategy, whose effectiveness is verified empirically through the ablation study in Section IV.

Driven by this insight, SS-RLPSO skips the spatially redundant evaluations and evaluates only a sparse subset of represen-

Algorithm 1: SS-RLPSO Framework

Input: Objective function f, dimension D, population size N, max evaluations $MaxFES$, search bounds $[\mathbf{Pop}_{\min}, \mathbf{Pop}_{\max}]$, velocity bounds $[\mathbf{VR}_{\min}, \mathbf{VR}_{\max}]$, sparse ratio $\rho = 0.1$
Output: Global best fitness $gbest_fit$, optimization history $history_gbest$

Initialize $\mathbf{X}, \mathbf{V} \in \mathbb{R}^{N \times D}$ uniformly within bounds;
Initialize $\mathbf{pbest} \leftarrow \mathbf{X}$, $pbest_fit \leftarrow \infty$;
Initialize $gbest_fit \leftarrow \infty$, $stag_count \leftarrow 0$;
Initialize Q-table $\mathbf{Q} \in \mathbb{R}^{9 \times 4}$, $FES \leftarrow 0$;
$K \leftarrow \max(1, \lceil \rho N \rceil)$;
$K_{\text{cvg}} \leftarrow \lfloor K/2 \rfloor$, $K_{\text{div}} \leftarrow K - K_{\text{cvg}}$;
for $i = 1$ **to** N **do**
  $pbest_fit(i) \leftarrow f(\mathbf{X}_i)$, $FES \leftarrow FES + 1$;
  if $pbest_fit(i) < gbest_fit$ **then**
    $gbest_fit \leftarrow pbest_fit(i)$, $\mathbf{g} \leftarrow \mathbf{pbest}_i$;
while $FES < MaxFES$ **do**
  $progress \leftarrow FES/MaxFES$;
  $s_{\text{phase}} \leftarrow$ PhaseState$(progress)$;
  $s_{\text{stag}} \leftarrow$ StagState$(stag_count)$;
  $s \leftarrow (s_{\text{phase}} - 1) \times 3 + s_{\text{stag}}$;
  $a \leftarrow \epsilon$-greedy$(\mathbf{Q}(s, :), \epsilon = 0.5 - 0.45 \times progress)$;
  $\mathbf{X}, \mathbf{V} \leftarrow$ OperatorUpdate$(\mathbf{X}, \mathbf{V}, \mathbf{pbest}, \mathbf{g}, a)$;
  $d_i^{\text{cvg}} \leftarrow \|\mathbf{X}_i - \mathbf{g}\|_2$, $\forall i$;
  $\mathcal{E}_{\text{cvg}} \leftarrow$ indices of the K_{cvg} smallest d_i^{cvg};
  $\bar{\mathbf{x}} \leftarrow \frac{1}{N}\sum_{i=1}^{N} \mathbf{X}_i$;
  $d_i^{\text{div}} \leftarrow \|\mathbf{X}_i - \bar{\mathbf{x}}\|_2$, $\forall i$;
  $\mathcal{E}_{\text{div}} \leftarrow$ indices of the K_{div} largest d_i^{div};
  $\mathcal{E} \leftarrow$ unique$(\mathcal{E}_{\text{cvg}} \cup \mathcal{E}_{\text{div}})$;
  $old_gbest \leftarrow gbest_fit$;
  for $i \in \mathcal{E}$ **do**
    $fit \leftarrow f(\mathbf{X}_i)$, $FES \leftarrow FES + 1$;
    if $fit < pbest_fit(i)$ **then**
      $pbest_fit(i) \leftarrow fit$, $\mathbf{pbest}_i \leftarrow \mathbf{X}_i$;
      if $fit < gbest_fit$ **then**
        $gbest_fit \leftarrow fit$, $\mathbf{g} \leftarrow \mathbf{X}_i$;
  $r \leftarrow$ Reward$(gbest_fit, old_gbest, s_{\text{stag}}, a)$;
  $stag_count \leftarrow$ UpdateStag$(gbest_fit, old_gbest)$;
  $s'_{\text{phase}} \leftarrow$ PhaseState$(FES/MaxFES)$;
  $s'_{\text{stag}} \leftarrow$ StagState$(stag_count)$;
  $s' \leftarrow (s'_{\text{phase}} - 1) \times 3 + s'_{\text{stag}}$;
  $\delta \leftarrow r + \gamma \max_{a'} \mathbf{Q}(s', a') - \mathbf{Q}(s, a)$;
  $\mathbf{Q}(s, a) \leftarrow \mathbf{Q}(s, a) + \alpha\delta$;
  $history_gbest(FES) \leftarrow gbest_fit$;
return $gbest_fit$, $history_gbest$

tative particles in each generation. Because concentrating all evaluations around the global best would quickly erode swarm diversity, the subset also covers particles located in sparsely visited regions. How this subset is built is described in the next subsection.

C. Spatial Screening Mechanism

The sparse evaluation framework is realized by the spatial screening mechanism. In each generation, all N particles update their positions according to the selected search operator, but only the screened particles receive real fitness evaluations, while the rest are shielded in that iteration. Position update is thus decoupled from fitness evaluation, which allows the population to stay large enough to preserve diversity while the evaluation cost remains controlled. The number of evaluated particles is governed by the sparse ratio ρ (empirically set to 0.1) and is limited to:

$$K = \max(1, \lceil \rho \times N \rceil) \tag{2}$$

To balance exploitation and exploration, this evaluation capacity is evenly divided into two groups:

$$K_{\text{cvg}} = \lfloor K/2 \rfloor, \quad K_{\text{div}} = K - K_{\text{cvg}} \tag{3}$$

1) Convergence-driven Exploitation Group: Particles close to the global best typically reside in promising local basins and are suitable for fine-grained exploitation. Since these particles have high potential to yield better solutions, allocating fitness evaluations to them helps the algorithm refine the current optimum and speed up convergence.

The exploitation potential of particle i is quantified by its Euclidean distance to the current global best position $\mathbf{g}$:

$$d_i^{\text{cvg}} = \|\mathbf{x}_i - \mathbf{g}\|_2 \tag{4}$$

The K_{cvg} particles with the smallest d_i^{cvg} are selected to form the exploitation set $\mathcal{E}_{\text{cvg}}$, aiming to refine the current optimal region.

2) Diversity-driven Exploration Group: Particles far from the population centroid generally lie in under-explored regions and help maintain population diversity. Evaluating these particles provides useful information about untested search areas, which is critical for preventing premature convergence to local optima.

The exploration potential is measured by the Euclidean distance from each particle to the population centroid $\bar{\mathbf{x}}$:

$$\bar{\mathbf{x}} = \frac{1}{N}\sum_{i=1}^{N} \mathbf{x}_i \tag{5}$$

$$d_i^{\text{div}} = \|\mathbf{x}_i - \bar{\mathbf{x}}\|_2 \tag{6}$$

The K_{div} particles with the largest d_i^{div} are selected to form the exploration set $\mathcal{E}_{\text{div}}$, aiming to preserve population diversity.

3) Evaluation Set Construction: The final evaluation set $\mathcal{E}$ is formed by merging the two groups and removing duplicate particles:

$$\mathcal{E} = \text{unique}\left(\mathcal{E}_{\text{cvg}} \cup \mathcal{E}_{\text{div}}\right) \tag{7}$$

Only particles in $\mathcal{E}$ undergo fitness evaluation. Due to potential overlap between the two groups, the actual number of evaluations satisfies $|\mathcal{E}| \leq K \ll N$.

The screening procedure involves only basic distance computation and sorting operations, leading to a per-generation algorithmic complexity of $O(N \log N + ND)$. Although this

mechanism reduces computational cost by skipping fitness evaluations for shielded particles, its intrinsic execution overhead is negligible relative to the cost of true function evaluations. Furthermore, in contrast to conventional surrogate-assisted methods that incur prohibitive overhead from $O(N_s^3)$ matrix inversion, our parameter-free geometric screening achieves savings in fitness evaluations with minimal additional computational cost, making it a lightweight approach for expensive optimization.

D. RL-Based Operator Scheduling

With most particles shielded from evaluation, the algorithm receives only sparse evolutionary feedback. Under such conditions, fixed operators may cause premature convergence. To address this, an RL agent is introduced to dynamically select search operators according to real-time evolutionary states.

The limited evaluation budget is thereby allocated to valuable particles through spatial screening, while the search behavior is adapted to the current evolutionary state.

1) State Space: To enable effective operator scheduling, the RL agent takes two complementary evolutionary characteristics into account: search progress and population stagnation status. Accordingly, the system state is defined as the composite vector $s = (s_{\text{phase}}, s_{\text{stag}})$, which forms a total of $3 \times 3 = 9$ discrete states.

The evolutionary phase indicator s_{phase} is designed to track the overall consumption of the computational budget. It allows the RL agent to adapt its search strategy over the optimization process, gradually shifting from global exploration to local exploitation as iterations progress. This indicator is derived from the evolutionary progress:

$$s_{\text{phase}} = \begin{cases} 1, & \textit{progress} < 0.3 \quad \text{(early stage)} \\ 2, & 0.3 \le \textit{progress} < 0.7 \quad \text{(mid stage)} \\ 3, & \textit{progress} \ge 0.7 \quad \text{(late stage)} \end{cases} \tag{8}$$

where $\textit{progress} = \textit{FEs}/\textit{MaxFEs}$.

Meanwhile, the stagnation indicator s_{stag} detects local optima trapping and provides a clear signal for the agent to invoke disruptive perturbation operators and escape stagnation. This state is determined by $\textit{stag_count}$, a counter tracking consecutive iterations without improvement in the global best fitness. Let Th_{stag} denote the stagnation detection threshold, fixed at 10 in this work. The stagnation state is defined as:

$$s_{\text{stag}} = \begin{cases} 1, & \textit{stag_count} = 0 \quad \text{(no stagnation)} \\ 2, & 1 \le \textit{stag_count} < Th_{\text{stag}} \quad \text{(mild stagnation)} \\ 3, & \textit{stag_count} \ge Th_{\text{stag}} \quad \text{(severe stagnation)} \end{cases} \tag{9}$$

The composite state index is computed as $s = (s_{\text{phase}} - 1) \times 3 + s_{\text{stag}}$, which maps each state pair to a unique integer in $[1, 9]$ and supports efficient Q-table indexing.

2) Action Space: The RL agent selects from an action space containing four dedicated topological search operators. For the first two operators, let $\mathbf{r}_1, \mathbf{r}_2 \sim U(0,1)^D$ denote two independent random vectors, and $\odot$ denote element-wise (Hadamard) multiplication.

OP1: Exploitation-Biased Update. This operator accelerates convergence by biasing particle motion toward personal and global best positions:

$$\begin{aligned} \mathbf{v}_i(t+1) = {} & 0.4\mathbf{v}_i(t) + 1.0\mathbf{r}_1 \odot (\mathbf{pbest}_i - \mathbf{x}_i(t)) \\ & + 2.5\mathbf{r}_2 \odot (\mathbf{g} - \mathbf{x}_i(t)) \end{aligned} \tag{10}$$

$$\mathbf{x}_i(t+1) = \mathbf{x}_i(t) + \mathbf{v}_i(t+1) \tag{11}$$

The larger weight on the global best term makes this operator well suited for fast local exploitation in stable convergence phases.

OP2: Peer Learning Update. This operator follows the comprehensive learning paradigm [11] to preserve population diversity:

$$\begin{aligned} \mathbf{v}_i(t+1) = {} & 0.9\mathbf{v}_i(t) + 1.5\mathbf{r}_1 \odot (\mathbf{pbest}_i - \mathbf{x}_i(t)) \\ & + 1.5\mathbf{r}_2 \odot (\mathbf{pbest}_r - \mathbf{x}_i(t)) \end{aligned} \tag{12}$$

$$\mathbf{x}_i(t+1) = \mathbf{x}_i(t) + \mathbf{v}_i(t+1) \tag{13}$$

where $r \in \{1, 2, \ldots, N\} \setminus \{i\}$ is a randomly selected peer index. This learning mechanism shares information among different particles and maintains population diversity.

OP3: Gaussian Perturbation. This operator abandons the velocity momentum term and imposes localized stochastic perturbations centered at the current global best $\mathbf{g}$:

$$\mathbf{x}_i(t+1) = \mathbf{g} + \boldsymbol{\epsilon}, \quad \boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \sigma^2 \mathbf{I}) \tag{14}$$

with an adaptively decaying standard deviation:

$$\sigma = 0.1 \cdot (1 - \textit{progress}) \cdot \|\mathbf{Pop}_{\max} - \mathbf{Pop}_{\min}\|_2 \tag{15}$$

where $\mathbf{Pop}_{\max}$ and $\mathbf{Pop}_{\min}$ are the upper and lower bounds of the search space. The velocity is reset to $\mathbf{v}_i(t+1) = \mathbf{0}$ to eliminate inertial drift.

OP4: Opposition-Based Global Jump. Executes an opposition-based learning strategy [43] to break out of local deadlocks:

$$\mathbf{x}_i(t+1) = \mathbf{Pop}_{\max} + \mathbf{Pop}_{\min} - \mathbf{x}_i(t) \tag{16}$$

$$\mathbf{v}_i(t+1) = -\mathbf{v}_i(t) \tag{17}$$

This operator provides a trajectory inversion, serving as a system reset mechanism when the swarm is trapped.

3) Reward Function and Q-Learning Update: The reward function is designed to balance short-term exploitation and long-term exploration during evolution:

$$r_t = \begin{cases} +10, & \text{if the fitness of } g \text{ improves,} \\ +0.5, & \text{if } s_{\text{stag}} = 3 \text{ and } a_t \in \{\text{OP3}, \text{OP4}\}, \\ -1, & \text{otherwise.} \end{cases} \tag{18}$$

When the population is in severe stagnation ($s_{\text{stag}} = 3$), OP1 and OP2 receive a reward of -1. To encourage escape from stagnation, the agent receives $+0.5$ for selecting OP3 or OP4, regardless of immediate fitness change. This prevents the policy from always selecting exploitation operators.

The Q-table is then updated by:

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha \left[r_t + \gamma \max_{a'} Q(s_{t+1}, a') - Q(s_t, a_t) \right] \tag{19}$$

using learning rate $\alpha = 0.1$, discount factor $\gamma = 0.9$, and an ϵ-greedy exploration rate linearly decaying via $\epsilon = 0.5 - (0.5 - 0.05) \cdot progress$. The bracketed term in (19) is the temporal-difference error, which Algorithm 1 computes separately as δ so that the same update can be written as $Q(s,a) \leftarrow Q(s,a) + \alpha\delta$.

E. Algorithm Procedure and Computational Efficiency

1) Time Complexity Analysis: The per-generation algorithmic overhead of SS-RLPSO constitutes three optimized components:

1) Virtual Particle Update: $O(ND)$, mathematically identical to canonical PSO.
2) Spatial Screening: $O(N \log N + ND)$, strictly dominated by computing distance arrays and identifying the boundary indices.
3) RL Scheduling: $O(1)$, executing a direct lookup on the 9×4 Q-table.

The total algorithmic overhead per generation is therefore:

$$T_{\text{SS-RLPSO}} = O(N \log N + ND) \tag{20}$$

This upper bound is independent of the cost of a single fitness evaluation. In contrast, the total time complexity of standard PSO is $T_{\text{PSO}} = O(N \cdot \mathcal{C})$, where $\mathcal{C}$ stands for the computational cost required for one fitness evaluation. In practical engineering optimization problems, $\mathcal{C}$ is frequently several orders of magnitude greater than $\log N + D$. When the evaluation cost $\mathcal{C}$ is high, the screening overhead $O(N \log N + ND)$ becomes nearly negligible, while our method avoids evaluating 90% of the particles per generation.

TABLE I: Comparison of Per-Generation Computational Complexity

Algorithm	Evaluations/Gen	Algorithmic Overhead
Canonical PSO	N	$O(ND)$
CLPSO / APSO	N	$O(ND)$
CCPSO2	N	$O(ND + T_{\text{group}})$
Surrogate-assisted	K_{surr}	$O(N_s^3)$
SSLPSO	$1 \sim 2$	$O(ND)$
SS-RLPSO	$\lceil \rho N \rceil$	$O(N \log N + ND)$

2) Comparison with Representative Algorithms: Table I compares the per-generation computational complexity of the proposed SS-RLPSO against a set of representative optimizers, covering canonical PSO variants, large-scale evolutionary algorithms, and surrogate-assisted methods. In the table, K_{surr} denotes the number of candidate solutions that the infill criterion of the surrogate model selects for real fitness evaluation in one generation, which is usually much smaller than the population size N, while N_s denotes the number of training samples used to build the surrogate model. Gaussian process-based surrogate models suffer from prohibitive $O(N_s^3)$ matrix inversion operations, which limit their scalability and confine them to low-dimensional expensive problems. In contrast, the geometric screening mechanism in SS-RLPSO maintains the overall overhead at $O(N \log N + ND)$, while retaining the ability to handle expensive large-scale optimization tasks. Moreover, unlike SSLPSO which relies on a static sparse framework, SS-RLPSO dynamically coordinates geometric screening and RL-based operator scheduling throughout the entire evolutionary process.

IV. Experiments and Discussion

A. Experimental Design and Setup

This section describes the benchmark suites, compared algorithms, parameter settings, and statistical procedures used to evaluate SS-RLPSO under both standard and resource-constrained budgets, including scalability tests in high-dimensional search spaces and supplementary studies on parameter sensitivity and module ablation.

First, baseline performance tests are conducted on the IEEE CEC2017 single-objective bound-constrained optimization suite [44] at $D = 50$ and $D = 100$. The compared algorithms include canonical PSO [2], CLPSO [11], HPSOT-VAC [25], and AWPSO [26], all of which are representative classical PSO variants following the full-population evaluation paradigm. To further examine the effectiveness of the RL-based operator scheduling under limited computational resources, SS-RLPSO is also tested with only 20% of the standard maximum fitness evaluations (MaxFEs).

To investigate the scalability of the sparse evaluation framework on complex high-dimensional landscapes, additional experiments are extended to the IEEE CEC2013 Large-Scale Global Optimization (LSGO) suite [45] with $D = 1000$. For a targeted comparison, the baselines in this part are algorithms specifically designed for large-scale optimization, including CSO [14], CCPSO2 [12], LSEO [24], and SSLPSO [34].

Beyond pairwise performance comparisons, two supplementary studies are performed to analyze the robustness and structural contributions of SS-RLPSO. A sensitivity analysis is carried out on four core hyperparameters: the sparse ratio ρ (i.e., the evaluation ratio K/N), the Q-learning learning rate α, the discount factor γ, and the stagnation detection threshold, to examine performance stability across different configurations. An ablation study is also conducted to quantify the individual contribution of the spatial screening mechanism and the RL-based scheduler, by comparing the full SS-RLPSO with three ablated variants: SS-RLPSO-w/o-SS (without spatial screening), SS-RLPSO-w/o-RL (without RL scheduler), and SS-RLPSO-w/o-Both (equivalent to canonical PSO). Both supplementary studies are conducted under the 20% MaxFEs setting to remain consistent with the resource-constrained optimization scenario.

All experiments are implemented in MATLAB R2024a and executed on a workstation with an Intel Core i5-13790F processor and 32 GB of RAM. For fair comparison, the population size is fixed at $N = 100$ for all experiments. The maximum number of fitness evaluations is set to $D \times 10{,}000$ for the CEC2017 suite, whereas the CEC2013 LSGO experiments adopt a micro-budget of $11 \times D = 11{,}000$ evaluations, uniformly applied to all compared methods. The internal parameters of each baseline algorithm are set according to the recommendations in their original publications. Each independent experiment is repeated 30 times to mitigate statistical

TABLE II: Optimization Results (Mean Best Fitness) on the CEC2017 Benchmark Suite at $D = 50$ and $D = 100$ under Standard MaxFEs

Func.	$D = 50$					$D = 100$				
	PSO	CLPSO	HPSOTVAC	AWPSO	**SS-RLPSO**	PSO	CLPSO	HPSOTVAC	AWPSO	**SS-RLPSO**
F1	1.30E+08+	4.11E+04−	**2.25E+03**−	2.38E+10+	6.22E+04	6.87E+08+	**3.27E+03**−	6.64E+03−	1.28E+11+	2.84E+05
F3	7.73E+02+	1.53E+05+	3.35E+02+	5.80E+04+	**3.21E+02**	9.07E+03+	4.56E+05+	1.54E+04+	3.86E+05+	**3.11E+03**
F4	5.89E+02+	5.39E+02−	**4.61E+02**−	3.20E+03+	5.65E+02	8.14E+02+	6.41E+02≈	**6.02E+02**−	1.97E+04+	6.49E+02
F5	7.49E+02+	7.07E+02+	7.61E+02+	7.67E+02+	**6.46E+02**	1.12E+03+	1.18E+03+	1.19E+03+	1.34E+03+	**8.53E+02**
F6	6.42E+02+	**6.00E+02**−	6.32E+02+	6.14E+02≈	6.07E+02	6.54E+02+	**6.00E+02**−	6.37E+02+	6.42E+02+	6.17E+02
F7	9.55E+02≈	9.77E+02≈	1.25E+03+	1.30E+03+	**9.32E+02**	1.54E+03+	1.48E+03+	2.56E+03+	4.00E+03+	**1.30E+03**
F8	1.05E+03+	1.01E+03+	1.07E+03+	1.08E+03+	**9.56E+02**	1.52E+03+	1.50E+03+	1.58E+03+	1.70E+03+	**1.16E+03**
F9	4.68E+03≈	3.97E+03+	8.64E+03+	8.07E+03+	**1.74E+03**	1.72E+04+	1.62E+04+	1.89E+04+	2.47E+04+	**5.99E+03**
F10	7.78E+03+	8.44E+03+	7.55E+03+	7.33E+03+	**6.22E+03**	1.54E+04+	2.33E+04+	1.54E+04+	1.63E+04+	**1.29E+04**
F11	1.31E+03−	1.31E+03−	**1.24E+03**−	2.03E+03+	1.43E+03	2.80E+03≈	6.12E+03+	**1.98E+03**−	4.49E+04+	2.67E+03
F12	2.66E+08+	1.45E+07−	**1.74E+05**−	5.60E+09+	1.54E+07	4.47E+08≈	2.50E+07−	**4.73E+05**−	3.27E+10+	4.78E+07
F13	1.07E+07+	5.44E+04−	**7.56E+03**−	1.18E+09+	2.56E+05	1.06E+06+	1.87E+04−	**1.04E+04**−	3.04E+09+	1.19E+05
F14	5.23E+04−	3.50E+05+	**1.75E+04**−	5.09E+05+	7.96E+04	1.08E+05−	3.52E+06+	**5.18E+04**−	1.27E+07+	6.84E+05
F15	7.91E+03−	**6.28E+03**−	8.25E+03−	4.77E+07≈	1.21E+05	3.41E+07+	5.26E+03−	**3.01E+03**−	1.35E+09+	1.11E+05
F16	3.16E+03≈	**2.72E+03**−	3.28E+03+	3.68E+03+	2.85E+03	5.53E+03+	5.06E+03+	5.94E+03+	7.51E+03+	**4.65E+03**
F17	3.05E+03≈	**2.52E+03**−	3.18E+03+	3.60E+03+	2.82E+03	5.34E+03+	**4.10E+03**−	5.48E+03+	9.41E+03+	4.66E+03
F18	1.61E+05−	1.11E+06≈	**4.24E+04**−	4.45E+06≈	6.61E+05	2.78E+05−	4.85E+06+	**1.25E+05**−	9.67E+06+	1.28E+06
F19	1.93E+04−	**2.65E+03**−	1.58E+04−	7.17E+07+	1.66E+05	9.78E+03−	4.01E+03−	**3.39E+03**−	1.04E+09+	6.86E+05
F20	2.97E+03≈	**2.58E+03**−	3.08E+03+	2.98E+03≈	2.77E+03	4.90E+03+	**4.20E+03**≈	5.11E+03+	5.19E+03+	4.46E+03
F21	2.57E+03+	2.52E+03+	2.64E+03+	2.61E+03+	**2.44E+03**	3.28E+03+	3.03E+03+	3.79E+03+	3.29E+03+	**2.69E+03**
F22	9.76E+03+	1.02E+04+	9.24E+03+	9.10E+03+	**7.97E+03**	1.87E+04+	2.55E+04+	1.85E+04+	1.84E+04+	**1.54E+04**
F23	3.22E+03+	2.95E+03+	3.85E+03+	3.11E+03+	**2.87E+03**	4.60E+03+	3.23E+03+	5.23E+03+	3.86E+03+	**3.18E+03**
F24	3.41E+03+	3.22E+03+	4.06E+03+	3.28E+03+	**3.03E+03**	5.48E+03+	3.88E+03+	6.09E+03+	4.54E+03+	**3.62E+03**
F25	3.07E+03+	3.08E+03+	3.07E+03+	4.91E+03+	**3.03E+03**	3.41E+03+	3.36E+03+	**3.28E+03**≈	1.87E+04+	3.31E+03
F26	7.53E+03+	5.90E+03+	1.12E+04+	7.72E+03+	**5.37E+03**	2.09E+04+	1.26E+04+	2.67E+04+	1.90E+04+	**9.70E+03**
F27	3.69E+03+	3.41E+03≈	5.13E+03+	3.61E+03+	**3.32E+03**	3.95E+03+	3.50E+03+	7.22E+03+	4.08E+03+	**3.45E+03**
F28	3.34E+03+	3.38E+03+	3.29E+03+	7.31E+03+	**3.28E+03**	3.56E+03+	3.54E+03+	**3.35E+03**−	2.02E+04+	3.40E+03
F29	4.71E+03+	**3.74E+03**−	5.08E+03+	4.96E+03+	4.23E+03	7.81E+03+	**5.73E+03**−	7.90E+03+	9.94E+03+	6.62E+03
+/−/≈	18 / 5 / 5	13 / 12 / 3	19 / 9 / 0	24 / 0 / 4	–	23 / 3 / 2	18 / 8 / 2	17 / 10 / 1	28 / 0 / 0	–
Time (s)	215.06	277.56	284.51	277.76	613.58	1426.95	1545.30	1556.06	1497.35	2546.79

Note: All fitness values are the mean best fitness over 30 independent runs, and the Time row reports the mean total CPU runtime. '+', '−', and '≈' indicate that SS-RLPSO is statistically better than, worse than, or similar to the compared peer algorithm, respectively, according to the Wilcoxon rank-sum test at a significance level of 0.05. Best results are highlighted in **bold**.

randomness. The Wilcoxon rank-sum test at the 0.05 significance level is employed to assess the statistical significance of performance differences [46].

B. Performance on the CEC2017 Benchmark Suite

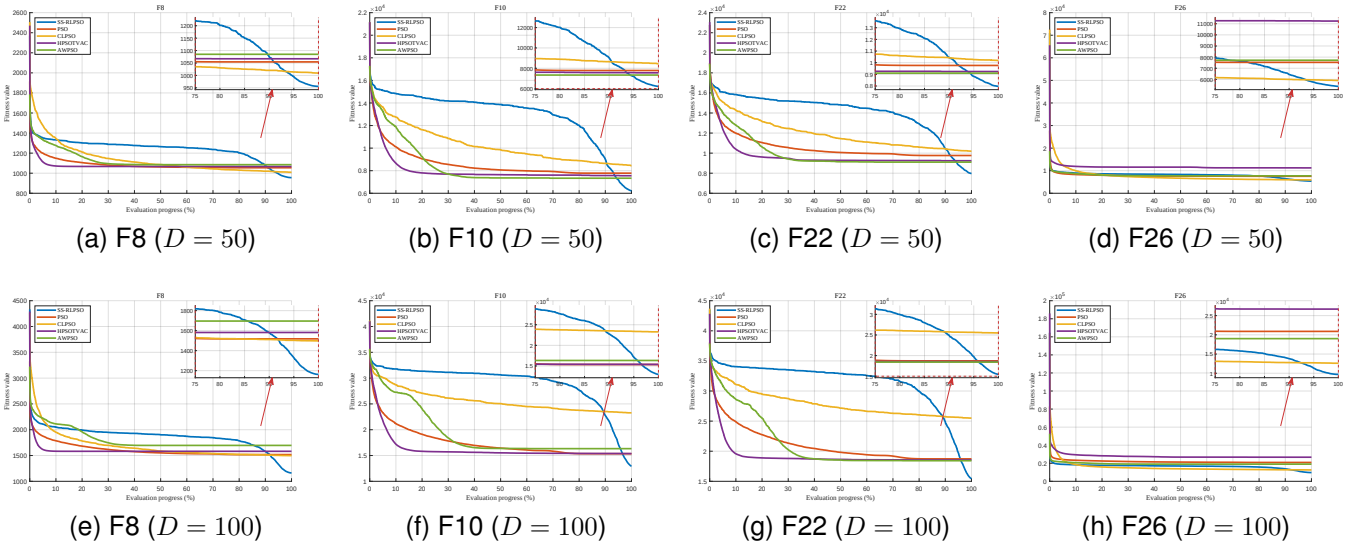
Table II summarizes the optimization accuracy and statistical results on the CEC2017 benchmark suite at $D = 50$ and $D = 100$ under the standard maximum fitness evaluations (MaxFEs). According to the Wilcoxon rank-sum test results (summarized as "$+ / - / \approx$"), SS-RLPSO outperforms the peer algorithms on most test functions. At $D = 100$, SS-RLPSO achieves 28 wins, 0 losses, and 0 ties against AWPSO, and 18 wins, 8 losses, and 2 ties against CLPSO.

Comparing the results across dimensions highlights the scalability of the proposed framework. For instance, although CLPSO remains competitive at $D = 50$ (SS-RLPSO holds only a narrow 13/12/3 edge over it), SS-RLPSO breaks this balance and widens the performance gap at $D = 100$. This suggests that the RL-based operator scheduling maintains search efficiency as the dimensionality increases.

The bottom rows of Table II detail the accumulated computational runtime under the standard MaxFEs. While SS-RLPSO consumes more CPU time than conventional PSOs due to internal distance calculations and Q-table updates, this runtime ratio between SS-RLPSO and its competitors narrows in higher dimensions. For example, the runtime ratio of SS-RLPSO to CLPSO decreases from approximately 2.2 at $D = 50$ to 1.6 at $D = 100$. This indicates that as the evaluation cost per solution scales with dimensionality, the internal algorithmic overhead becomes less significant. Therefore, for expensive optimization problems where fitness evaluation dominates the total cost, this runtime trade-off is justified by the improved solution quality.

To provide a more intuitive understanding of the evolutionary dynamics, Fig. 2 illustrates the convergence profiles of the compared algorithms on four representative test functions (F8, F10, F22, and F26). A comparative analysis between the top row ($D = 50$) and the bottom row ($D = 100$) reveals a clear discrepancy in algorithmic scalability. At $D = 50$, advanced variants such as CLPSO exhibit fast initial convergence and occasionally match the final accuracy of the proposed algorithm. However, as the problem dimensionality expands to $D = 100$, the exponential growth of the search space limits their exploratory capabilities, causing most peer algorithms to suffer from premature convergence and stagnate early in the evolutionary process.

In contrast, SS-RLPSO demonstrates robustness against stagnation, which is visually evidenced by the distinctive sharp drops in its convergence curves during the later evaluation stages. This phenomenon is most prominent in complex multimodal landscapes, such as F8 and F10 at $D = 100$. When the swarm converges to local optima and standard search operators lose their effectiveness, the RL-based operator scheduler detects the stagnation state and adaptively triggers



(a) F8 ($D = 50$) (b) F10 ($D = 50$) (c) F22 ($D = 50$) (d) F26 ($D = 50$)

(e) F8 ($D = 100$) (f) F10 ($D = 100$) (g) F22 ($D = 100$) (h) F26 ($D = 100$)

Fig. 2: Convergence profiles of SS-RLPSO and peer algorithms on representative CEC2017 test functions. The top row shows the results at $D = 50$, and the bottom row shows the results at $D = 100$.

TABLE III: Optimization Accuracy and Execution Time on the CEC2013 LSGO Suite ($D = 1000$) under $11,000$ MaxFEs (mean values over 30 independent runs)

Func.	Fitness Results					Total CPU Runtime (s)				
	CSO	CCPSO2	LSEO	SSLPSO	**SS-RLPSO**	CSO	CCPSO2	LSEO	SSLPSO	**SS-RLPSO**
F1	4.31E+10+	**7.20E+05**−	7.40E+09−	1.67E+10−	2.38E+10	2.16	499.41	4651.48	80.99	8.13
F2	3.19E+04−	**2.08E+01**−	2.29E+04−	1.93E+04−	3.95E+04	2.47	631.27	4623.79	78.01	8.43
F3	2.08E+01−	**3.97E-02**−	2.04E+01−	1.90E+01−	2.11E+01	0.38	55.92	4701.01	117.19	4.23
F4	1.64E+12+	**1.44E+11**−	9.81E+11−	1.46E+12+	1.33E+12	2.76	729.41	4608.88	75.90	9.60
F5	1.23E+07≈	1.78E+07+	2.29E+07+	**1.08E+07**−	1.18E+07	3.70	952.66	4569.39	76.53	11.30
F6	6.24E+05+	9.84E+05+	9.84E+05+	**4.12E+05**≈	5.14E+05	0.60	112.23	4775.25	113.16	5.32
F7	4.31E+10+	**3.03E+09**−	7.51E+09−	8.51E+09−	1.04E+10	3.31	843.79	4555.21	75.67	10.85
F8	3.89E+16+	**4.63E+15**−	4.27E+16+	3.06E+16+	1.26E+16	3.96	1009.48	4469.71	72.95	11.74
F9	9.68E+08−	1.17E+09≈	1.78E+09+	**8.53E+08**−	1.06E+09	5.39	1451.75	4494.50	75.14	15.07
F10	1.72E+07+	7.11E+07+	8.40E+07+	1.60E+07+	**1.39E+07**	0.99	182.87	4712.26	98.66	5.66
F11	1.68E+13+	**3.08E+11**−	1.00E+12−	1.15E+12−	2.66E+12	9.72	2466.91	4336.46	80.39	23.62
F12	1.13E+12+	**4.80E+03**−	2.44E+11−	1.85E+11−	9.64E+11	0.25	48.65	4710.55	123.83	3.95
F13	6.14E+12+	**2.29E+10**−	7.97E+10−	3.35E+11≈	4.14E+11	9.65	2482.15	3692.62	72.72	23.45
F14	1.85E+13+	**4.59E+11**−	1.07E+12−	2.73E+12−	4.09E+12	10.19	2585.99	3750.59	73.99	23.45
F15	6.06E+13+	**1.20E+08**−	1.44E+11−	3.60E+11−	2.39E+12	9.09	2421.68	4245.21	79.75	21.95
+/−/≈	11 / 3 / 1	3 / 11 / 1	5 / 10 / 0	3 / 10 / 2	–	–	–	–	–	–

Note: All fitness values are the mean best fitness over 30 independent runs, and the runtime columns report mean total CPU times. '+', '−', and '≈' indicate that SS-RLPSO is statistically better than, worse than, or similar to the compared algorithm, respectively, according to the Wilcoxon rank-sum test at a significance level of 0.05. Best fitness values are highlighted in **bold**.

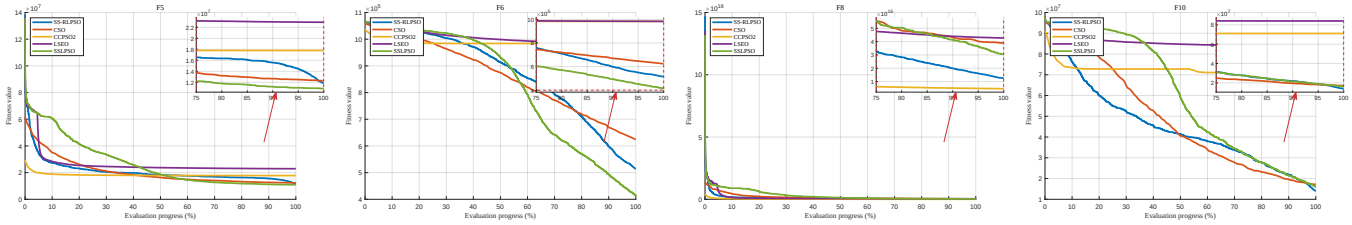
the perturbation operators (i.e., OP3 and OP4). These targeted perturbations reinvigorate the population's diversity, allowing the swarm to escape local optima and continue refining the solutions until the evaluation budget is exhausted.

Overall, SS-RLPSO attains competitive accuracy under the standard budget and scales better with dimensionality than the peer PSO variants. Its accuracy–efficiency trade-off and behavior under restricted budgets are examined in the following subsections.

C. Scalability Analysis on the CEC2013 LSGO Benchmark

To evaluate SS-RLPSO on high-dimensional problems with limited budgets, this section conducts experiments on the CEC2013 LSGO suite with $D = 1000$. Following the micro-budget setting used in LSEO [24], the maximum number of fitness evaluations is constrained to $11 \times D = 11{,}000$. This budget is smaller than standard LSGO protocols and is specifically designed to simulate computationally expensive optimization scenarios. Table III reports the final optimization accuracy and CPU runtimes.

In terms of accuracy, SS-RLPSO outperforms CSO with a record of 11/3/1 under this restricted budget. It also achieves competitive results on several functions (e.g., F5, F6, F8, and F10). However, in terms of pure final accuracy, it is outperformed by specialized large-scale optimizers such as CCPSO2 (3/11/1) and SSLPSO (3/10/2). This accuracy gap stems from fundamentally different design objectives rather



(a) F5 (b) F6 (c) F8 (d) F10

Fig. 3: Convergence profiles of SS-RLPSO and peer algorithms on representative CEC2013 LSGO test functions ($D = 1000$). The evolutionary trajectories are recorded under the micro-budget setting of $11,000$ MaxFEs. Despite lacking specialized vector decomposition mechanisms, SS-RLPSO maintains a competitive descent trajectory in complex landscapes (e.g., F10).

than a deficiency of the proposed framework. CCPSO2 and LSEO rely on cooperative coevolution, variable grouping, or surrogate modeling to decompose 1000-dimensional problems, while SSLPSO introduces statically designed selection indicators to refine accuracy. SS-RLPSO, by contrast, is designed as a lightweight optimizer whose primary objective is execution speed under micro-budgets. The mechanistic difference from SSLPSO is also evident: SSLPSO updates at most two representative individuals per generation according to predefined heuristic selection rules, whereas SS-RLPSO evaluates a geometrically screened subset of about 10% of the population and dynamically schedules four search operators via the RL agent, which preserves richer population feedback at a much lower selection cost.

Fig. 3 shows the convergence profiles on four representative functions. CSO drops quickly at first but stagnates early. CCPSO2 and LSEO descend steadily over a long period. SS-RLPSO maintains a continuous search trajectory throughout the limited 11,000 evaluations, despite having no vector decomposition mechanism. On F10, it avoids local optima and achieves the best final accuracy.

The specialized architectures of the baseline algorithms, however, carry a heavy runtime cost. As shown in the right panel of Table III, while CCPSO2 and LSEO require hundreds to thousands of seconds due to their complex internal mechanisms, SS-RLPSO completes each run in 4 to 24 seconds, which is close to CSO and several times faster than SSLPSO (about 70–124 seconds).

These results reveal a clear accuracy–time trade-off in large-scale optimization. In latency-sensitive engineering scenarios, such as online control, embedded parameter tuning, or digital-twin-based decision making, an optimizer that requires hundreds to thousands of seconds of internal computation per run is impractical regardless of its final accuracy, since the decision window is typically far shorter than its running time. SS-RLPSO offers a pragmatic alternative at a different point of this trade-off: it works directly on the full high-dimensional vector without expensive decomposition or surrogate modeling, and delivers acceptable accuracy with a runtime that is orders of magnitude shorter. For applications where a timely, reasonably good solution is more valuable than a slightly better but late one, this trade-off is precisely the design goal of the proposed framework.

D. Performance under Restricted Evaluation Budgets

TABLE IV: Statistical Summary and Runtime of SS-RLPSO (20% MaxFEs) against Peer Algorithms (Full MaxFEs) (mean values over 30 independent runs)

Dim.	Metric	Peer Algorithms (100% MaxFEs)				**SS-RLPSO**
		PSO	CLPSO	HPSOTVAC	AWPSO	(20% **MaxFEs**)
$D = 50$	$+ / - / \approx$	16/7/5	12/12/4	16/11/1	23/0/5	–
	Time (s)	207.64	252.24	255.10	237.44	**114.94**
$D = 100$	$+ / - / \approx$	15/6/7	12/13/3	13/12/3	28/0/0	–
	Time (s)	1345.36	1458.96	1476.39	1443.19	**481.10**

Note: All reported values are the mean over 30 independent runs. The peer-algorithm runtimes were recorded in an independent set of runs and may differ slightly from those in Table II. '$+$', '$-$', and '$\approx$' indicate that SS-RLPSO is statistically better than, worse than, or similar to the peer algorithm, respectively, according to the Wilcoxon rank-sum test at a significance level of 0.05.

The previous section evaluated SS-RLPSO under the standard MaxFEs budget. In practice, however, the evaluation budget is often limited by expensive evaluation cost or time constraints, making full-evaluation methods impractical. To examine SS-RLPSO under such conditions, this section allocates only one-fifth of the standard MaxFEs to SS-RLPSO, while all peer algorithms use the full budget. The statistical results and CPU runtimes on the CEC2017 suite at $D = 50$ and $D = 100$ are summarized in Table IV.

Despite the reduced budget, SS-RLPSO achieves competitive results. At $D = 50$, it significantly outperforms AWPSO with 23 wins and 0 losses, and performs comparably to CLPSO with a record of 12/12/4. At $D = 100$, SS-RLPSO continues to outperform AWPSO (28/0/0), while remaining competitive with CLPSO (12/13/3) and HPSOTVAC (13/12/3). These results indicate that the RL-based operator scheduling adapts search behaviors to different evolutionary states, directing the limited evaluation budget toward valuable particles.

The reduced evaluation budget also leads to shorter runtimes. By skipping about 90% of the per-generation fitness evaluations through spatial screening, SS-RLPSO requires less CPU time than its competitors. At $D = 50$, SS-RLPSO takes only 114.94 seconds, compared with 252.24 seconds for CLPSO. At $D = 100$, SS-RLPSO requires 481.10 seconds, while the competing algorithms consume between 1345 and 1476 seconds. This shows that the screening overhead is offset by the reduction in fitness evaluations, making SS-RLPSO

efficient for expensive optimization problems. This runtime advantage follows from the reduced evaluation budget assigned to SS-RLPSO in this setting; under an equal full budget (Table II), its internal screening and Q-learning overhead makes it slower than the peer algorithms.

E. Sensitivity Investigation of Parameters

TABLE V: Wilcoxon Rank-Sum Test Results ($+/-/\approx$) of SS-RLPSO with Different Hyperparameter Settings against the Default Configuration under a Constrained Computational Budget (20% MaxFEs) over 30 independent runs

Parameter Category	**Tested Value**	**Result** ($+/-/\approx$)
Evaluation Ratio (K/N) (Default: 0.1)	0.06	$1/0/27$
	0.20	$0/0/28$
	0.30	$1/1/26$
	0.50	$0/5/23$
Learning Rate (α) (Default: 0.1)	0.3	$1/0/27$
	0.5	$1/2/25$
	0.7	$0/1/27$
	0.9	$2/0/26$
Discount Factor (γ) (Default: 0.9)	0.3	$0/2/26$
	0.5	$1/1/26$
	0.7	$1/2/25$
	0.99	$3/1/24$
Stagnation Threshold (Th_{stag}) (Default: 10)	5	$1/0/27$
	20	$0/1/27$
	30	$2/2/24$
	50	$1/1/26$

Note: '+', '−', and '≈' denote that the variant performs significantly better than, significantly worse than, and similar to the default configuration, respectively.

To examine the robustness of SS-RLPSO against hyperparameter variations, a sensitivity analysis is conducted on the CEC2017 benchmark suite under the constrained setting of 20% standard MaxFEs. Four core parameters are investigated: the sparse ratio ρ, the Q-learning learning rate α, the discount factor γ, and the stagnation detection threshold. Each parameter is tested across multiple candidate values, and the Wilcoxon rank-sum test at the 0.05 significance level is used to compare performance against the default configuration. The statistical results are summarized in Table V. The evaluation ratio (K/N) is the most sensitive parameter. As the ratio increases from the default 0.1 to 0.5, the algorithm degrades significantly, with 5 test functions showing statistically worse results. This is consistent with the design rationale: under a severely constrained budget, evaluating a large proportion of redundant particles rapidly exhausts the available FEs and curtails the number of evolutionary generations. By skipping 90% of the particle evaluations ($K/N = 0.1$), SS-RLPSO preserves the FE budget to extend the evolutionary process, which also gives the RL scheduler enough episodes to converge. The learning rate α, the discount factor γ, and the stagnation threshold (Th_{stag}) are comparatively robust: most variants yield statistical ties, and although an extreme value like $\gamma = 0.99$ shows a slight advantage (3 wins) due to a longer planning horizon, the default configurations remain competitive without problem-specific tuning. This indicates that the advantage of SS-RLPSO comes from its architecture rather than from finely tuned parameters.

TABLE VI: Wilcoxon Rank-Sum Test Results ($+/-/\approx$) of the Proposed SS-RLPSO against its Ablation Variants under a Constrained Computational Budget (20% MaxFEs, 30 independent runs)

Ablation Variant	**Result** ($+/-/\approx$)
SS-RLPSO-w/o-SS (Without Spatial Screening)	$10/2/16$
SS-RLPSO-w/o-RL (Without RL Scheduler)	$7/3/18$
SS-RLPSO-w/o-Both (Canonical PSO Baseline)	$26/0/2$

Note: '+', '−', and '≈' denote that the proposed SS-RLPSO performs significantly better than, significantly worse than, and similar to the compared variant, respectively.

F. Ablation Study under Limited Computational Budget

To examine the contribution of each component under a strict budget, we conducted an ablation study. As in the previous experiments, the maximum number of fitness evaluations was limited to 20% of the standard setting. Three variants were compared: (1) SS-RLPSO-w/o-SS, which disables the spatial screening to perform full-population evaluations; (2) SS-RLPSO-w/o-RL, which replaces the RL scheduler with random operator selection; and (3) SS-RLPSO-w/o-Both, representing the canonical baseline PSO. The statistical results are summarized in Table VI. The detailed mean and standard deviation values, along with the convergence curves of representative test functions, are provided in the supplementary material.

The ablation results confirm the essential role of the spatial screening (SS) mechanism. Removing SS forces the algorithm to evaluate the entire population each generation, which quickly exhausts the limited FE budget and results in premature termination. As a result, the full SS-RLPSO outperforms this variant on 10 of the 28 test functions. The exclusion of the RL scheduler also leads to performance degradation, with 7 losses against the complete algorithm. This indicates that the sparse feedback arising from spatial screening makes operator selection challenging without an adaptive scheduler. It should be noted that the RL scheduler and random operator selection show no statistically significant difference on the remaining 18 functions. This suggests that the primary performance gain of the framework stems from the spatial screening mechanism, while the RL scheduler provides complementary improvements mainly on complex multimodal functions, where stagnation-aware perturbation is critical, as visually evidenced by the sharp drops in the convergence curves in Fig. 2. A deeper analysis of the learned scheduling policy, together with comparisons against fixed-rule schedulers, will be pursued in future work. The canonical PSO baseline, which lacks both mechanisms, is outperformed on 26 functions, suggesting that conventional swarm intelligence algorithms are not well-suited for strict computational budgets without explicit redesign.

V. Conclusion

This paper presents a Spatial Screening and Reinforcement Learning-Based Particle Swarm Optimization (SS-RLPSO) to address the limitation of the conventional full-population evaluation paradigm, which restricts the deployment of swarm intelligence algorithms in real-time or resource-constrained engineering applications. Departing from this paradigm, the proposed algorithm adopts a model-free sparse evaluation framework. A lightweight spatial screening mechanism is designed to identify and skip fitness evaluations for spatially redundant particles, preserving the limited evaluation budget to sustain more evolutionary generations. To compensate for the information loss caused by sparse evaluation, an RL-based operator scheduler is integrated, which adjusts the search strategy according to the evolutionary phase and stagnation status.

Extensive experiments on the CEC2017 suite and the CEC2013 large-scale global optimization (LSGO) suite ($D = 1000$) validate the accuracy–efficiency trade-off of SS-RLPSO. Under the standard full budget, it attains accuracy competitive with representative PSO variants, and its advantage becomes more pronounced as the dimensionality increases. When restricted to only 20% of the standard budget, it still delivers accuracy comparable to that of full-budget peer algorithms at roughly one-third to one-half of their wall-clock time, and on 1000-dimensional problems it runs more than two orders of magnitude faster than the surrogate-assisted optimizer LSEO. The ablation study attributes the primary gain to spatial screening, with the RL scheduler providing complementary improvements on complex multimodal functions, while the sensitivity analysis indicates that the performance of the framework stems from its architecture rather than from finely tuned parameters.

Future work will extend this research in three directions. First, the spatial screening mechanism will be adapted to multi-objective optimization scenarios, with a focus on identifying redundant individuals along the Pareto front. Second, lightweight surrogate models will be incorporated to provide low-cost fitness estimation for unevaluated particles, further improving search efficiency under sparse evaluation budgets. Third, the proposed algorithm will be applied to computationally expensive engineering tasks, such as aerodynamic design optimization and robotic trajectory planning.

References

[1] A. E. Eiben, J. E. Smith *et al.*, *Introduction to evolutionary computing*. Springer, 2003, vol. 53.

[2] J. Kennedy and R. Eberhart, "Particle swarm optimization," in *Proceedings of ICNN'95-international conference on neural networks*, vol. 4. ieee, 1995, pp. 1942–1948.

[3] D. Wang, D. Tan, and L. Liu, "Particle swarm optimization algorithm: an overview," *Soft Comput.*, vol. 22, no. 2, pp. 387–408, 2018.

[4] Z.-H. Zhan, J. Zhang, Y. Li, and H. S.-H. Chung, "Adaptive particle swarm optimization," *IEEE Trans. Syst. Man Cybern. B, Cybern.*, vol. 39, no. 6, pp. 1362–1381, 2009.

[5] R. Storn and K. Price, "Differential evolution–a simple and efficient heuristic for global optimization over continuous spaces," *J. Glob. Optim.*, vol. 11, no. 4, pp. 341–359, 1997.

[6] D. Karaboğa and S. Ökdem, "A simple and global optimization algorithm for engineering problems: differential evolution algorithm," *Turk. J. Electr. Eng. Comput. Sci.*, vol. 12, no. 1, pp. 53–60, 2004.

[7] F. Neri and V. Tirronen, "Recent advances in differential evolution: a survey and experimental analysis," *Artif. Intell. Rev.*, vol. 33, no. 1, pp. 61–106, 2010.

[8] R.-J. Wai, J.-D. Lee, and K.-L. Chuang, "Real-time pid control strategy for maglev transportation system via particle swarm optimization," *IEEE Trans. Ind. Electron.*, vol. 58, no. 2, pp. 629–646, 2010.

[9] J. Bi, H. Yuan, S. Duanmu, M. Zhou, and A. Abusorrah, "Energy-optimized partial computation offloading in mobile-edge computing with genetic simulated-annealing-based particle swarm optimization," *IEEE Internet Things J.*, vol. 8, no. 5, pp. 3774–3785, 2020.

[10] V. Roberge, M. Tarbouchi, and G. Labonté, "Comparison of parallel genetic algorithm and particle swarm optimization for real-time uav path planning," *IEEE Trans. Ind. Informat.*, vol. 9, no. 1, pp. 132–141, 2012.

[11] J. J. Liang, A. K. Qin, P. N. Suganthan, and S. Baskar, "Comprehensive learning particle swarm optimizer for global optimization of multimodal functions," *IEEE Trans. Evol. Comput.*, vol. 10, no. 3, pp. 281–295, 2006.

[12] X. Li, X. Yao *et al.*, "Cooperatively coevolving particle swarms for large scale optimization." *IEEE Trans. Evol. Comput.*, vol. 16, no. 2, pp. 210–224, 2012.

[13] W. H. Lim and N. A. M. Isa, "Teaching and peer-learning particle swarm optimization," *Appl. Soft Comput.*, vol. 18, pp. 39–58, 2014.

[14] R. Cheng and Y. Jin, "A competitive swarm optimizer for large scale optimization," *IEEE Trans. Cybern.*, vol. 45, no. 2, pp. 191–204, 2014.

[15] X. Zhang, X. Zheng, R. Cheng, J. Qiu, and Y. Jin, "A competitive mechanism based multi-objective particle swarm optimizer with fast convergence," *Inf. Sci.*, vol. 427, pp. 63–76, 2017.

[16] Y. Tian, X. Li, H. Ma, X. Zhang, K. C. Tan, and Y. Jin, "Deep reinforcement learning based adaptive operator selection for evolutionary multi-objective optimization," *IEEE Trans. Emerg. Top. Comput. Intell.*, vol. 7, no. 4, pp. 1051–1064, 2022.

[17] S. Yin, M. Jin, H. Lu, G. Gong, W. Mao, G. Chen, and W. Li, "Reinforcement-learning-based parameter adaptation method for particle swarm optimization," *Complex Intell. Syst.*, vol. 9, no. 5, pp. 5585–5609, 2023.

[18] S. Shao, Y. Tian, S. Yang, and X. Zhang, "Deep reinforcement learning-assisted automated operator portfolio for constrained multi-objective optimization," *IEEE Trans. Emerg. Top. Comput. Intell.*, 2026.

[19] B. Liu, Q. Zhang, and G. G. Gielen, "A gaussian process surrogate model assisted evolutionary algorithm for medium scale expensive optimization problems," *IEEE Trans. Evol. Comput.*, vol. 18, no. 2, pp. 180–192, 2013.

[20] Y. Jin, "Surrogate-assisted evolutionary computation: Recent advances and future challenges," *Swarm Evol. Comput.*, vol. 1, no. 2, pp. 61–70, 2011.

[21] ——, "A comprehensive survey of fitness approximation in evolutionary computation," *Soft Comput.*, vol. 9, no. 1, pp. 3–12, 2005.

[22] R. G. Regis, "Particle swarm with radial basis function surrogates for expensive black-box optimization," *J. Comput. Sci.*, vol. 5, no. 1, pp. 12–23, 2014.

[23] M. Seeger, "Gaussian processes for machine learning," *Int. J. Neural Syst.*, vol. 14, no. 02, pp. 69–106, 2004.

[24] K. Zhao, X. Wang, C. Sun, Y. Jin, and A. Hayat, "Efficient large-scale expensive optimization via surrogate-assisted sub-problem selection," *IEEE Trans. Evol. Comput.*, 2025.

[25] A. Ratnaweera, S. K. Halgamuge, and H. C. Watson, "Self-organizing hierarchical particle swarm optimizer with time-varying acceleration coefficients," *IEEE Trans. Evol. Comput.*, vol. 8, no. 3, pp. 240–255, 2004.

[26] W. Liu, Z. Wang, Y. Yuan, N. Zeng, K. Hone, and X. Liu, "A novel sigmoid-function-based adaptive weighted particle swarm optimizer," *IEEE Trans. Cybern.*, vol. 51, no. 2, pp. 1085–1093, 2019.

[27] D. Tian, Q. Xu, X. Yao, G. Zhang, Y. Li, and C. Xu, "Diversity-guided particle swarm optimization with multi-level learning strategy," *Swarm Evol. Comput.*, vol. 86, p. 101533, 2024.

[28] X. Yang and H. Li, "Evolutionary-state-driven multi-swarm cooperation particle swarm optimization for complex optimization problem," *Inf. Sci.*, vol. 646, p. 119302, 2023.

[29] H. Moazen, S. Molaei, L. Farzinvash, and M. Sabaei, "Pso-elpm: Pso with elite learning, enhanced parameter updating, and exponential mutation operator," *Inf. Sci.*, vol. 628, pp. 70–91, 2023.

[30] Y. Tang, K. Huang, Z. Tan, M. Fang, and H. Huang, "Multi-subswarm cooperative particle swarm optimization algorithm and its application," *Inf. Sci.*, vol. 677, p. 120887, 2024.

[31] Y. Tian, L. Si, X. Zhang, R. Cheng, C. He, K. C. Tan, and Y. Jin, "Evolutionary large-scale multi-objective optimization: A survey," *ACM Comput. Surv.*, vol. 54, no. 8, pp. 1–34, 2021.

[32] R. Cheng and Y. Jin, "A social learning particle swarm optimization algorithm for scalable optimization," *Inf. Sci.*, vol. 291, pp. 43–60, 2015.
[33] Q. Yang, W.-N. Chen, T. Gu, H. Zhang, J. D. Deng, Y. Li, and J. Zhang, "Segment-based predominant learning swarm optimizer for large-scale optimization." *IEEE Trans. Cybern.*, vol. 47, no. 9, pp. 2896–2910, 2017.
[34] S. Liu, Z.-J. Wang, Z. Kou, Z.-H. Zhan, S. Kwong, and J. Zhang, "Less is more: A small-scale learning particle swarm optimization for large-scale optimization," *IEEE Trans. Cybern.*, 2025.
[35] S. Qin and C. Sun, "Expensive multiobjective optimization with adaptive contribution sampling and enhanced search," *IEEE Trans. Syst. Man Cybern. Syst.*, 2025.
[36] J. Liang, Y. Lou, M. Yu, Y. Bi, and K. Yu, "A survey of surrogate-assisted evolutionary algorithms for expensive optimization," *J. Membr. Comput.*, vol. 7, no. 2, pp. 108–127, 2025.
[37] Y. Liang, Z. Ren, Y. Yang, A. Chen, D. Guo, and B. Pang, "A mahalanobis distance-based fitness approximation method for estimation of distribution algorithms in solving expensive optimization problems," in *2019 IEEE International Conference on Systems, Man and Cybernetics (SMC)*. IEEE, 2019, pp. 1608–1613.
[38] P. Dayan and C. Watkins, "Q-learning," *Mach. Learn.*, vol. 8, no. 3, pp. 279–292, 1992.
[39] Y. Song, Y. Wu, Y. Guo, R. Yan, P. N. Suganthan, Y. Zhang, W. Pedrycz, S. Das, R. Mallipeddi, O. S. Ajani *et al.*, "Reinforcement learning-assisted evolutionary algorithm: A survey and research opportunities," *Swarm Evol. Comput.*, vol. 86, p. 101517, 2024.
[40] G. Liu, X. Su, R. Zhou, and Y. Tian, "Reinforcement learning-based adaptive particle swarm optimization," *Complex Intell. Syst.*, 2026.
[41] F. Wang, X. Wang, and S. Sun, "A reinforcement learning level-based particle swarm optimization algorithm for large-scale optimization," *Inf. Sci.*, vol. 602, pp. 298–312, 2022.
[42] X. Wang, F. Wang, Q. He, and Y. Guo, "A multi-swarm optimizer with a reinforcement learning mechanism for large-scale optimization," *Swarm Evol. Comput.*, vol. 86, p. 101486, 2024.
[43] H. R. Tizhoosh, "Opposition-based learning: a new scheme for machine intelligence," in *International conference on computational intelligence for modelling, control and automation and international conference on intelligent agents, web technologies and internet commerce (CIMCA-IAWTIC'06)*, vol. 1. IEEE, 2005, pp. 695–701.
[44] G. Wu, R. Mallipeddi, and P. Suganthan, "Problem definitions and evaluation criteria for the cec 2017 competition and special session on constrained single objective real-parameter optimization," Nanyang Technological University, Singapore, Tech. Rep., 2016.
[45] J. J. Liang, B. Y. Qu, P. N. Suganthan, and A. G. Hernández-Díaz, "Problem definitions and evaluation criteria for the cec 2013 special session on real-parameter optimization," Computational Intelligence Laboratory, Zhengzhou University, Zhengzhou, China and Nanyang Technological University, Singapore, Tech. Rep. 201212, 2013.
[46] J. Derrac, S. García, D. Molina, and F. Herrera, "A practical tutorial on the use of nonparametric statistical tests as a methodology for comparing evolutionary and swarm intelligence algorithms," *Swarm Evol. Comput.*, vol. 1, no. 1, pp. 3–18, 2011.